

AI ASSISTANT CODING LAB-11.4

Name:P.Chithra

Batch:41

Rollno:2303a52137

Task 1: Stack for Undo Operations (LIFO)

Prompt Used

Design a Python class that models an Undo feature for a simple text editor using the Stack data structure. The class should include methods to add a new action, remove the latest action, view the most recent action without deleting it, and check whether any actions exist. Provide meaningful docstrings, demonstrate its use with test cases, justify why the stack structure is appropriate for undo systems, and discuss the differences between implementing the stack using a Python list and using the collections.deque module.

Code

```
stack.py > UndoStack > latest
1 class UndoStack:
2     """
3     It stores user actions and allows reversal of the most
4     recent action using the LIFO (Last In, First Out) principle.
5     """
6
7     def __init__(self):
8         """Create an empty container to store actions."""
9         self._history = []
10
11     def add_action(self, action):
12         """
13         Record a new action.
14
15         Parameters:
16         action (str): Description of the operation performed.
17         """
18         self._history.append(action)
19
20     def undo(self):
21         """
22         Reverse and return the latest action.
23
24         Returns:
25         str: Last recorded action.
26
27         Raises:
28         RuntimeError: If no actions are available.
29         """
30         if self.empty():
31             raise RuntimeError("Nothing to undo.")
32         return self._history.pop()
33
34     def latest(self):
35         """
36         Display the most recent action without removing it.
37
38         Returns:
39         str or None: Latest action if available.
40         """
41         return None if self.empty() else self._history[-1]
42
43     def empty(self):
44         """
45         Check whether any actions exist.
46         """
```

```

27         str: Last recorded action.
28
29     Raises:
30         RuntimeError: If no actions are available.
31     """
32     if self.empty():
33         raise RuntimeError("Nothing to undo.")
34     return self._history.pop()
35
36     def latest(self):
37         """
38         Display the most recent action without removing it.
39
40         Returns:
41             str or None: Latest action if available.
42         """
43         return None if self.empty() else self._history[-1]
44
45     def empty(self):
46         """
47         Check whether any actions exist.
48
49         Returns:
50             bool: True if history is empty.
51         """
52         return len(self._history) == 0
53
54 if __name__ == "__main__":
55     editor = UndoStack()
56
57     # User activity simulation
58     editor.add_action("Inserted paragraph")
59     editor.add_action("Changed font size")
60     editor.add_action("Applied italics")
61
62     print("Most recent:", editor.latest())
63
64     # Undo operations
65     print("Undo operation:", editor.undo())
66     print("Undo operation:", editor.undo())
67
68     print("Remaining latest:", editor.latest())

```

Optimized

```
stack.py > ...
1  # ----- List Stack -----
2  class Stack:
3      def __init__(self):
4          self.items = []
5
6      def push(self, x): self.items.append(x)
7      def pop(self): return self.items.pop() if self.items else None
8      def peek(self): return self.items[-1] if self.items else None
9      def is_empty(self): return not self.items
10
11
12  print("Stack using list:")
13  s = Stack()
14  s.push("Type A")
15  s.push("Type B")
16  s.push("Delete B")
17
18  print("Undo:", s.pop())
19  print("Top:", s.peek())
20  print("Undo:", s.pop())
21
22
23  # ----- Deque Stack -----
24  from collections import deque
25
26  class StackDeque:
27      def __init__(self):
28          self.items = deque()
29
30      def push(self, x): self.items.append(x)
31      def pop(self): return self.items.pop() if self.items else None
32      def peek(self): return self.items[-1] if self.items else None
33      def is_empty(self): return not self.items
34
35
36  print("\nStack using deque:")
37  d = StackDeque()
38  d.push("Type X")
39  d.push("Type Y")
40  d.push("Delete Y")
41
42  print("Undo:", d.pop())
43  print("Top:", d.peek())
44  print("Undo:", d.pop())
45
```

Output:

```
D:\Downloads\New folder>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/New folder/stack.py"
D:\Downloads\New folder>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/New folder/stack.py"
Most recent: Applied italics
Undo operation: Applied italics
Undo operation: Changed font size
Remaining latest: Inserted paragraph
oads/New folder/stack.py"
Most recent: Applied italics
Undo operation: Applied italics
Undo operation: Changed font size
Remaining latest: Inserted paragraph
```

```
Undo: Type B
D:\Downloads\New folder>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/New folder/stack.py"
Stack using list:
Undo: Delete B
Top: Type B
Undo: Type B
Undo: Delete B
Top: Type B
Undo: Type B
Top: Type B
Undo: Type B
Undo: Type B

Stack using deque:
Undo: Delete Y
Top: Type Y
Undo: Type Y
```

Justification

Why Stack is Suitable for Undo

Undo operations must reverse the most recent action first. This matches LIFO behavior:

- Latest action → undone first
- Earlier actions → undone later

Example:

1. Type A
2. Type B
3. Delete B

Undo → Delete B undone first

Stack provides $O(1)$ push and pop, making it ideal for frequent undo operations.

Task 2: Queue for Customer Service (FIFO)

Prompt Used

Design a customer support system in Python where service requests are handled in the order they arrive. Implement a Queue class with methods to add a request, process the next request, and check whether the queue is empty. First use a list-based approach. Then analyze its performance, suggest an optimized version using collections.deque, and explain why the deque structure is more efficient in real-world applications.

Code:

```
stack.py > ...
1 class CustomerQueue:
2     """
3     Queue implementation for customer service requests.
4     Follows FIFO (First In, First Out) principle.
5     """
6
7     def __init__(self):
8         self.requests = []
9
10    def enqueue(self, request):
11        """Add a new customer request."""
12        self.requests.append(request)
13
14    def dequeue(self):
15        """Process the oldest request."""
16        if self.is_empty():
17            return "No requests"
18        return self.requests.pop(0)
19
20    def is_empty(self):
21        """Check if queue is empty."""
22        return len(self.requests) == 0
23
24
25 # ----- Testing List Queue -----
26 print("Customer Queue using list:")
27
28 q = CustomerQueue()
29 q.enqueue("Password Reset")
30 q.enqueue("Billing Issue")
31 q.enqueue("Technical Support")
32
33 print("Serving:", q.dequeue())
34 print("Serving:", q.dequeue())
35 print("Is empty?", q.is_empty())
36
```

Optimized

```
stack.py > ...
1  from collections import deque
2  class CustomerDequeQueue:
3      """
4      Optimized queue using deque for better performance.
5      """
6
7      def __init__(self):
8          self.requests = deque()
9
10     def enqueue(self, request):
11         self.requests.append(request)
12
13     def dequeue(self):
14         if self.is_empty():
15             return "No requests"
16         return self.requests.popleft()
17
18     def is_empty(self):
19         return len(self.requests) == 0
20
21
22     # ----- Testing Deque Queue -----
23     print("\nCustomer Queue using deque:")
24
25     dq = CustomerDequeQueue()
26     dq.enqueue("Account Unlock")
27     dq.enqueue("Refund Request")
28     dq.enqueue("Login Issue")
29
30     print("Serving:", dq.dequeue())
31     print("Serving:", dq.dequeue())
32     print("Is empty?", dq.is_empty())
33
```

Output:

Undo: Type Y

[Open file in editor \(ctrl + click\)](#)

```
D:\Downloads\New folder>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/New folder/stack.py"
Customer Queue using list:
Serving: Password Reset
Serving: Billing Issue
Is empty? False

D:\Downloads\New folder>
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER 1

```
D:\Downloads\New folder>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/New folder/stack.py"
Customer Queue using deque:
Serving: Account Unlock
Serving: Refund Request
Is empty? False
```


Performance Analysis

List-Based Queue

- enqueue $\rightarrow O(1)$
- dequeue $\rightarrow O(n)$ (shifts elements)

For 10,000 requests, dequeue becomes expensive.

Deque-Based Queue

- enqueue $\rightarrow O(1)$
- dequeue $\rightarrow O(1)$

Deque uses a doubly-linked structure internally, avoiding element shifting.

Real-World Impact

In high-volume systems (banking, server queues, chat systems), list-based queues degrade performance significantly. Deque ensures stable $O(1)$ operations.

Task 3: Singly Linked List for Dynamic Playlist Management

Prompt and code

```
queue.py > ...
1 #Implement a Queue in Python to handle customer service requests in FIFO order.
2 # First create a list-based queue with enqueue, dequeue, and is_empty methods. Then
3 # then implement an optimized version using collections.deque and compare performance.
4 # List-based Queue
5 class Queue:
6     def __init__(self):
7         self.data = []
8
9     def enqueue(self, x): self.data.append(x)
10    def dequeue(self): return self.data.pop(0) if self.data else None
11    def is_empty(self): return not self.data
12
13
14 print("List Queue:")
15 q = Queue()
16 q.enqueue("Reset Password")
17 q.enqueue("Billing Issue")
18
19 print("Serving:", q.dequeue())
20 print("Empty?", q.is_empty())
21
```

Output:

```
D:\Downloads\DSA codes>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/DSA codes/queue.py"
List Queue:
Serving: Reset Password
Empty? False

D:\Downloads\DSA codes>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/DSA codes/queue.py"
List Queue:
Serving: Reset Password
Empty? False

D:\Downloads\DSA codes>
```

Edge Cases

1. Empty list insertion
2. Delete from empty list
3. Delete head node
4. Delete single node list
5. Delete non-existent value

Task 4: Binary Search Tree for Fast Record Lookup

Scenario

You are building a student record system where quick searching by roll number is required.

Task Description

- Implement a Binary Search Tree (BST) with:
 - o insert(value)
 - o search(value)
 - o inorder_traversal()
- Provide AI with a partially written Node and BST class.

Prompt used :

Complete missing methods, Add meaningful docstrings, Explain how BST improves search efficiency over linear search

Code

```
queue.py > ...
1 class Node:
2     """
3     Represents a single node in the Binary Search Tree.
4     Each node contains data and pointers to left and right children.
5     """
6
7     def __init__(self, value):
8         self.value = value
9         self.left = None
10        self.right = None
11
12
13 class BST:
14     """Binary Search Tree implementation."""
15
16     def __init__(self):
17         self.root = None
18
19     def insert(self, value):
20         """
21         Insert a value into the BST.
22         Maintains BST property: left < root < right.
23         """
24         if not self.root:
25             self.root = Node(value)
26         else:
27             self._insert(self.root, value)
28
29     def _insert(self, node, value):
30         """Recursive helper for insertion."""
31         if value < node.value:
32             if node.left:
33                 self._insert(node.left, value)
34             else:
35                 node.left = Node(value)
```

```

29     def _insert(self, node, value):
30         """Recursive helper for insertion."""
31         if value < node.value:
32             if node.left:
33                 self._insert(node.left, value)
34             else:
35                 node.left = Node(value)
36         else:
37             if node.right:
38                 self._insert(node.right, value)
39             else:
40                 node.right = Node(value)
41
42     def search(self, value):
43         """
44         Search for a value in the BST.
45
46         Returns:
47         bool: True if found, otherwise False.
48         """
49         return self._search(self.root, value)
50
51     def _search(self, node, value):
52         """Recursive helper for search."""
53         if not node:
54             return False
55         if node.value == value:
56             return True
57         elif value < node.value:
58             return self._search(node.left, value)
59         else:
60             return self._search(node.right, value)
61
62     def inorder(self):
63         """Inorder traversal of BST."""
64         self._inorder(self.root)
65         print()
66
67     def _inorder(self, node):
68         """Helper for inorder traversal."""

```

queue.py > ...

```
13 class BST:
51     def _search(self, node, value):
52         """Recursive helper for search."""
53         if not node:
54             return False
55         if node.value == value:
56             return True
57         elif value < node.value:
58             return self._search(node.left, value)
59         else:
60             return self._search(node.right, value)
61
62     def inorder(self):
63         """Inorder traversal of BST."""
64         self._inorder(self.root)
65         print()
66
67     def _inorder(self, node):
68         """Helper for inorder traversal."""
69         if node:
70             self._inorder(node.left)
71             print(node.value, end=" ")
72             self._inorder(node.right)
73
74
75 # ----- Testing -----
76 if __name__ == "__main__":
77     tree = BST()
78
79     tree.insert(50)
80     tree.insert(30)
81     tree.insert(70)
82     tree.insert(20)
83     tree.insert(40)
84
85     print("Inorder traversal:")
86     tree.inorder()
87
88     print("Search 40:", tree.search(40))
89     print("Search 90:", tree.search(90))
90
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

SPELL CHECKER 9

Output:

```
D:\Downloads\DSA codes>C:/Users/Chithra/AppData/Local/Programs/Python/Python313/python.exe "d:/Downloads/DSA codes/queue.py"
Inorder traversal:
20 30 40 50 70
Search 40: True
Search 90: False
Inorder traversal:
20 30 40 50 70
Search 40: True
Search 90: False
Search 40: True
Search 90: False
Search 90: False
```

Why BST Improves Search Efficiency

Approach	Time Complexity
----------	-----------------

Linear Search	$O(n)$
---------------	--------

BST (Average)	$O(\log n)$
---------------	-------------

BST (Worst - Skewed)	$O(n)$
----------------------	--------

BST reduces search space by half at each step in balanced cases.

Task 5:

Graph Traversal for Social Network Connections

Scenario

You are modeling a social network, where users are connected to friends, and you want to explore connections.

Task Description

Represent the network using a graph (adjacency list).

- Use AI to implement:

- o Breadth-First Search (BFS) to find nearby connections

- o Depth-First Search (DFS) to explore deep connection paths

prompt used:

Add inline comments explaining traversal steps, Compare recursive and iterative DFS approaches, Suggest practical use cases for BFS vs DFS

Code:

```
queue.py > SocialNetwork
1 class SocialNetwork:
2     def __init__(self):
3         self.graph = {}
4
5     def add_friend(self, u, v):
6         # Add users if not present
7         if u not in self.graph:
8             self.graph[u] = []
9         if v not in self.graph:
10            self.graph[v] = []
11
12        # Create two-way friendship
13        self.graph[u].append(v)
14        self.graph[v].append(u)
15
16        # ----- BFS -----
17        def bfs(self, start):
18            visited = set()
19            queue = [start]
20
21            while queue:
22                current = queue.pop(0) # FIFO
23                if current not in visited:
24                    print(current, end=" ")
25                    visited.add(current)
26
27                # Add friends to queue
28                for f in self.graph[current]:
29                    if f not in visited:
30                        queue.append(f)
31            print()
32
33        # ----- DFS (recursive) -----
34        def dfs(self, node, visited=None):
35            if visited is None:
36                visited = set()
37
38            print(node, end=" ")
39            visited.add(node)
```

```

1  class SocialNetwork:
17     def bfs(self, start):
21         queue = deque()
22         current = queue.pop(0) # FIFO
23         if current not in visited:
24             print(current, end=" ")
25             visited.add(current)
26
27         # Add friends to queue
28         for f in self.graph[current]:
29             if f not in visited:
30                 queue.append(f)
31         print()
32
33     # ----- DFS (recursive) -----
34     def dfs(self, node, visited=None):
35         if visited is None:
36             visited = set()
37
38         print(node, end=" ")
39         visited.add(node)
40
41         # Visit deeper friends
42         for f in self.graph[node]:
43             if f not in visited:
44                 self.dfs(f, visited)
45
46
47     # ----- Test -----
48     sn = SocialNetwork()
49     sn.add_friend("A", "B")
50     sn.add_friend("A", "C")
51     sn.add_friend("B", "D")
52     sn.add_friend("C", "E")
53
54     print("BFS:")
55     sn.bfs("A")
56
57     print("DFS:")
58     sn.dfs("A")
59

```

Output:

```

DFS:
A B D C E
D:\Downloads\DSA codes>

```


BFS vs DFS Comparison

Feature	BFS	DFS
Structure	Queue	Stack/Recursion
Traversal	Level-by-level	Depth-first
Shortest Path	Yes (unweighted)	No
Memory	Higher (wide graphs)	Lower (deep graphs)